Unit 7: Backtracking and Greedy Approaches: Games and Compression

EXPLORING HEURISTIC PROBLEM-SOLVING TECHNIQUES

Ivan Temesvari
Distinguished Professor of Mathematics and Computer Science
Oakton College

Introduction

Problem-Solving Paradigms

Backtracking:

- Systematically explores all possible solutions by incrementally building candidates and abandoning those that lead to dead ends.

Greedy Algorithms:

- Make locally optimal choices at each step, hoping to find a global optimum

Applications:

- Backtracking: Puzzles, constraint satisfaction problems, game playing
- Greedy Algorithms: Optimization problems, data compression, scheduling

Huffman Coding

Problem: Lossless data compression, representing characters using fewer bits

Greedy Approach:

Assign shorter codes to more frequent characters, longer codes to less frequent ones.

Build a Huffman tree based on character frequencies

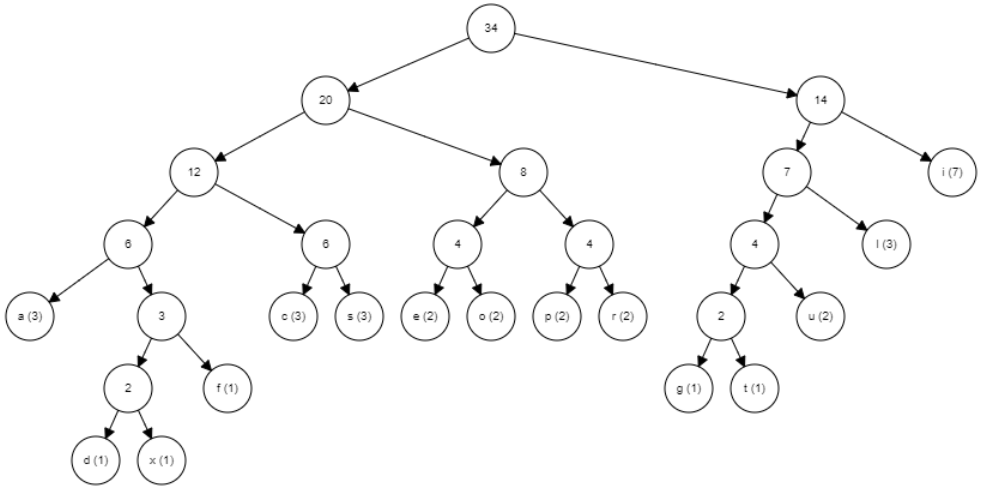
Algorithm Steps:

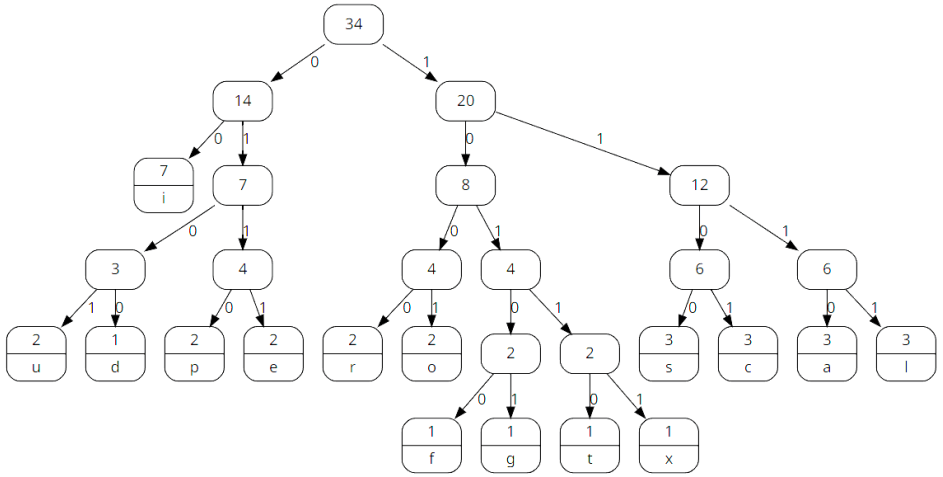
1. Build a frequency table for characters in the input
2. Create a priority queue (min-heap) of nodes, each representing a character and its frequency
3. Repeatedly extract two minimum frequency nodes, combine them into a new node with their frequencies summed, and insert the new node back into the priority queue
4. The final node in the priority queue represents the root of the Huffman tree
5. Traverse the tree to assign codes to each character (0 for left branch, 1 for right branch)

Huffman Coding - Example

Illustrative example:

- Show a small text string and its character frequencies: **`supercalifragilisticexpialidocious`**
- Demonstrate the construction of the Huffman tree step-by-step
- Show the resulting Huffman codes for each character
- Calculate the compression ratio





Animation Completed

8-Queens Chess Problem

Problem: Place 8 queens on a chessboard so that no two queens threaten each other (i.e., no two queens share the same row, column, or diagonal)

Backtracking Approach:

- Try placing a queen in each column, one by one
- If a placement is safe (doesn't conflict with previously placed queens), move to the next column
- If a placement is unsafe, backtrack to the previous column and try a different position

8-Queens - Backtracking Visualization

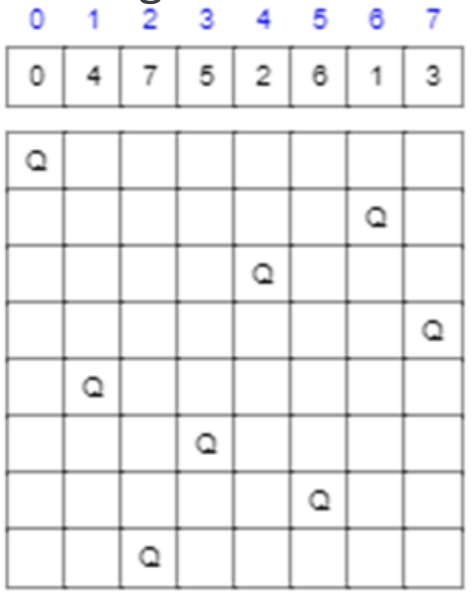
Illustrate the backtracking process:

Show a chessboard and the placement of queens

Highlight the current column being considered

Show how the algorithm explores different positions in the column, backtracking when necessary

Demonstrate a successful solution where all 8 queens are placed safely



Code Examples (Pseudocode)

Example 1: Huffman Coding

```
FUNCTION build_huffman_tree(frequencies)
    priority_queue = new PriorityQueue()
    FOR EACH character, frequency IN frequencies
        node = new Node(character, frequency)
        priority_queue.enqueue(node)
    WHILE priority_queue.size() > 1
        left = priority_queue.dequeue()
        right = priority_queue.dequeue()
        new_node = new Node(null, left.frequency + right.frequency)
        new_node.left = left
        new_node.right = right
        priority_queue.enqueue(new_node)
    RETURN priority_queue.dequeue()  // Root of the Huffman tree
// ... (Additional functions for assigning codes and encoding/decoding data)
```

Code Examples (Pseudocode)

Example 2: 8-Queens Problem

```
FUNCTION solve_8_queens(board, col)
    IF col == 8
        RETURN true  // All queens placed successfully
    FOR EACH row from 0 to 7
        IF is_safe(board, row, col)
            board[row][col] = 1  // Place queen
            IF solve_8_queens(board, col + 1)
                RETURN true
            board[row][col] = 0  // Backtrack: remove queen
    RETURN false  // No safe position in this column


FUNCTION is_safe(board, row, col)
 // Check if the queen is safe in the given row and column
 // ... (Check row, column, and diagonals for conflicts)
```

Case Studies

Data Compression: Huffman coding is widely used in file compression formats (e.g., ZIP, JPEG, MP3) to reduce file sizes.

Game Playing: Backtracking algorithms are used in game AI to explore possible moves and find optimal strategies (e.g., chess, sudoku).

Resource Allocation: Greedy algorithms can be applied to problems like task scheduling and resource allocation to find efficient solutions.

Conclusion

Key Takeaways

- Backtracking and greedy algorithms are powerful problem-solving techniques.
- Huffman coding achieves efficient data compression using a greedy approach
- The 8-queens problem demonstrates the application of backtracking for constraint satisfaction
- Understanding these techniques enables you to tackle a wide range of problems in various domains